

## Beginner: Syntax and Basic Literals

1. Write a function that takes an integer `status_code` and uses `match-case` to return "OK" for 200, "Not Found" for 404, "Server Error" for 500, and "Unknown" for anything else.
2. Create a script that accepts a string `day` and returns "Weekend" if it matches "Saturday" or "Sunday" (using a single `case` with the `|` operator), and "Weekday" otherwise.
3. Write a basic text-adventure function that takes a string command ("look", "walk", "inventory"). Use a wildcard (`_`) to return "Invalid command" if the user types anything else.
4. Write a function that takes a single character. Use `match-case` to return "Vowel" if it is 'a', 'e', 'i', 'o', or 'u', and "Consonant" for all other letters.
5. Create a function that accepts a string `operator` ("add", "subtract", "multiply", "divide") and returns the corresponding mathematical symbol ("+", "-", "\*", "/").
6. Write a function that matches a boolean variable `is_active`. Print "Account Active" if True, and "Account Inactive" if False.
7. Create a script that takes an integer `menu_choice` from 1 to 3. Match the numbers to execute placeholder print statements like "Starting game...", "Loading save...", and "Exiting...", handling invalid integers gracefully.
8. Write a function that takes a string `direction` ("N", "S", "E", "W") and returns the full word ("North", "South", "East", "West") using `match-case`.

## Intermediate: Sequence Unpacking, Dictionaries, and Guards

9. Write a function that takes a list of two numbers `[x, y]` representing a 2D coordinate. Use `match` to print "Origin" if `[0, 0]`, "On X-axis" if `[x, 0]`, "On Y-axis" if `[0, y]`, and "Point (x, y)" for anything else.
10. Create a pattern match that accepts a list of unknown length. Match when the first element is `"error"`, capture all remaining elements into a variable `details` using the `*` (star) operator, and print them.
11. Write a function that takes a dictionary `{"user": "...", "role": "..."}.` Match cases specifically for `{"role": "admin"}` and `{"role": "guest"}`, printing different welcome messages.
12. Create a function that uses a **guard** (`if` condition) inside a `case` to match a tuple `(name, age)`. Match only if `age >= 18` and return "Adult: [name]".
13. Write a function that accepts a single argument of an unknown built-in type. Match against the types `int`, `str`, and `list` (e.g., `case str(s):`), returning a formatted string describing the type and value.
14. Create a program that takes a dictionary payload. Match exactly when it contains both `"latitude"` and `"longitude"` keys, capture their values, and return a formatted GPS string.

15. Write a function matching a tuple `(action, amount)`. If action is "withdraw", use guards to check if `amount` is positive and less than 1000. If it is negative, trigger a different case.
16. Create a function that matches a sequence `["log", level, message]`. Match specifically when `level` is "CRITICAL", capturing the `message` to return a high-priority alert string.
17. Write a pattern that unpacks a nested dictionary `{"response": {"status": int, "data": list}}`. Match only when the status is 200 and extract the `data` list into a variable.

### Advanced: Classes, Deep Nesting, and Aliasing

18. Define a dataclass `User(name: str, age: int, role: str)`. Write a `match-case` block that matches a `User` instance specifically when the role is "admin", capturing the `name` attribute.
19. Create two classes, `Circle(radius)` and `Rectangle(width, height)`. Write a function that accepts an object and uses `match` to calculate and return the area based on which class instance is passed.
20. Write a recursive function to parse a simple Abstract Syntax Tree (AST) represented by nested tuples, e.g., `("add", ("num", 5), ("num", 3))`, evaluating and returning the final mathematical result using `match`.
21. Create a dictionary pattern that uses the `as` keyword to match `{"user": {"id": 123, "permissions": ["read", "write"]}}`. Capture the entire `{"id": 123, ...}` sub-dictionary as `user_data` while simultaneously validating its internal structure.
22. Write a function that matches a complex JSON-like list. Match when the *first* item is a dictionary with `{"type": "header"}` and capture the *last* item in the list into a variable, ignoring everything in the middle using `*middle`.
23. Define a custom Python class `Color` and manually implement the `__match_args__` dunder attribute. Write a `match` statement that successfully unpacks positional arguments `(r, g, b)` from a `Color` instance.
24. Write a function that takes a nested list representing a shell command pipeline, e.g., `["grep", "error", ["tail", "-n", 10]]`. Use structural pattern matching to extract and print the inner command only if the outer command is "grep".
25. Combine type matching, sequence unpacking, and dictionary matching: Write a single `match` block that handles a webhook payload that could be either a raw string (log message), a list of strings (multiple logs), or a structured dictionary `{"error": code, "trace": [...]}`.